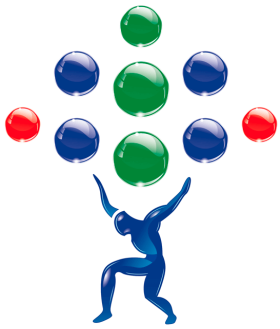
UNIVERSIDAD PRIVADA DOMINGO SAVIO



ACTIVIDAD 2: EVOLUCIÓN DEL CATÁLOGO WEB: PHP, ARQUITECTURA MVC, MYSQL, PDO Y CRUD SEGURO.

Carrera: Ingeniería de Sistemas

Asignatura: Tecnología Web II

Docente: M.Sc. Ing. Jimmy Nataniel Requena Llorentty

Integrantes:

Alejandro Fabián Cabrera Flores

María Alejandra Suárez O.

Jade Guissel Flores Coria

Ariel Salas Tococari

Renzo Molina Navia

Santa Cruz de la Sierra - octubre 2026

ÍNDICE

RESUMEN 3

ABSTRACT 3

1. INTRODUCCIÓN 5

2. OBJETIVOS 5

2.1. Objetivo General 5

2.2. Objetivos Específicos 5

3. DESARROLLO POR LABORATORIOS 6

3.1. Laboratorio 6: Poo, Composer y Polimorfismo 6

3.2. Laboratorio 7: Front Controller, Router Y Seguridad De Vistas 8

3.3. Laboratorio 8: Mysql, Pdo Y Crud De Productos 9

3.4. Laboratorio 9: Repositorios, Socios, Filtros Y Paginación 11

4. ELEMENTOS TÉCNICOS COMUNES 15

4.1. Acceso a Datos con PDO 15

4.2. Consultas preparadas e inyección SQL 15

4.3. Validación, integridad y errores 15

5. RESUMEN DE RESULTADOS DOCUMENTADOS 15

6. ANÁLISIS 16

7. CONCLUSIONES 17

8. RECOMENDACIONES 17

REFERENCIAS 18

RESUMEN

El informe describe la evolución de una aplicación web de catálogo desarrollada en PHP para la gestión de productos y servicios apícolas. Los laboratorios 6 y 7 fortalecen la programación orientada a objetos, la carga automática de clases, el enrutamiento y el manejo seguro de errores. Los laboratorios 8 y 9 llevan la aplicación hacia MySQL mediante PDO, consultas preparadas, operaciones CRUD, relaciones entre productos y socios, filtros y paginación. Las bitácoras aportan evidencias de verificadores satisfactorios: 14/14 en el laboratorio 6, 15/15 en el laboratorio 7, 15/15 en el laboratorio 8 y 14/14 en el laboratorio 9. En seguridad, la evidencia del laboratorio 9 muestra una búsqueda con una entrada de prueba similar a una inyección SQL; su captura debe interpretarse junto con el código parametrizado y las comprobaciones del laboratorio.

Palabras clave: PHP, POO, MVC, MySQL, PDO, CRUD, consultas preparadas, seguridad web, paginación.

ABSTRACT

This report describes the evolution of a PHP-based web catalog application designed to manage beekeeping products and services. Laboratories 6 and 7 focus on object-oriented programming, automatic class loading, request routing, and secure error handling. Laboratories 8 and 9 extend the application to MySQL using PDO, prepared statements, CRUD operations, relationships between products and partners, filtering, and pagination. The laboratory logs provide evidence of successful verification tests, reporting 14 out of 14 checks in Laboratory 6, 15 out of 15 in Laboratory 7, 15 out of 15 in Laboratory 8, and 14 out of 14 in Laboratory 9. Regarding security, the evidence from Laboratory 9 includes a search using an input designed to test SQL injection resistance. This evidence must be considered alongside the parameterized query



implementation and the verification results to assess the application's protection against SQL injection.

Keywords: PHP, object-oriented programming, MVC, MySQL, PDO, CRUD, prepared statements, web security, pagination.

1. INTRODUCCIÓN

La aplicación de catálogo requiere representar productos y servicios, mostrar información en el navegador, recibir formularios y conservar los datos de manera consistente. A medida que aumenta la funcionalidad, se vuelve importante separar las responsabilidades del modelo, el controlador, las vistas y la capa de acceso a datos. También es necesario validar las entradas, gestionar errores con respuestas HTTP apropiadas y evitar que los datos enviados por los usuarios modifiquen las consultas SQL.

La Actividad 2 reúne los aprendizajes desarrollados en los laboratorios 6 al 9. El recorrido va desde la organización orientada a objetos y Composer hasta la integración con MySQL, la implementación de operaciones de administración y la presentación de listados con filtros y paginación.

2. OBJETIVOS

2.1. Objetivo General

Migrar la aplicación de catálogo a MySQL y completar el CRUD de productos y socios mediante PDO y consultas preparadas, implementando el modelo de datos, las relaciones entre tablas, la validación y la paginación, y comprobando la resistencia de la aplicación ante intentos de inyección SQL.

2.2. Objetivos Específicos

I. Aplicar Composer y PSR-4 para organizar y cargar clases automáticamente.

II. Utilizar clases abstractas, herencia, interfaces, traits, enums y polimorfismo en el modelo de productos y servicios.

III. Centralizar las solicitudes web mediante un front controller y un router, con excepciones y códigos HTTP adecuados.

IV. Diseñar y utilizar tablas relacionadas para productos y socios en MySQL.

V. Separar los valores de entrada de las instrucciones SQL mediante consultas preparadas de PDO.

VI. Implementar operaciones CRUD, validación, filtros, ordenamiento y paginación.

3. DESARROLLO POR LABORATORIOS

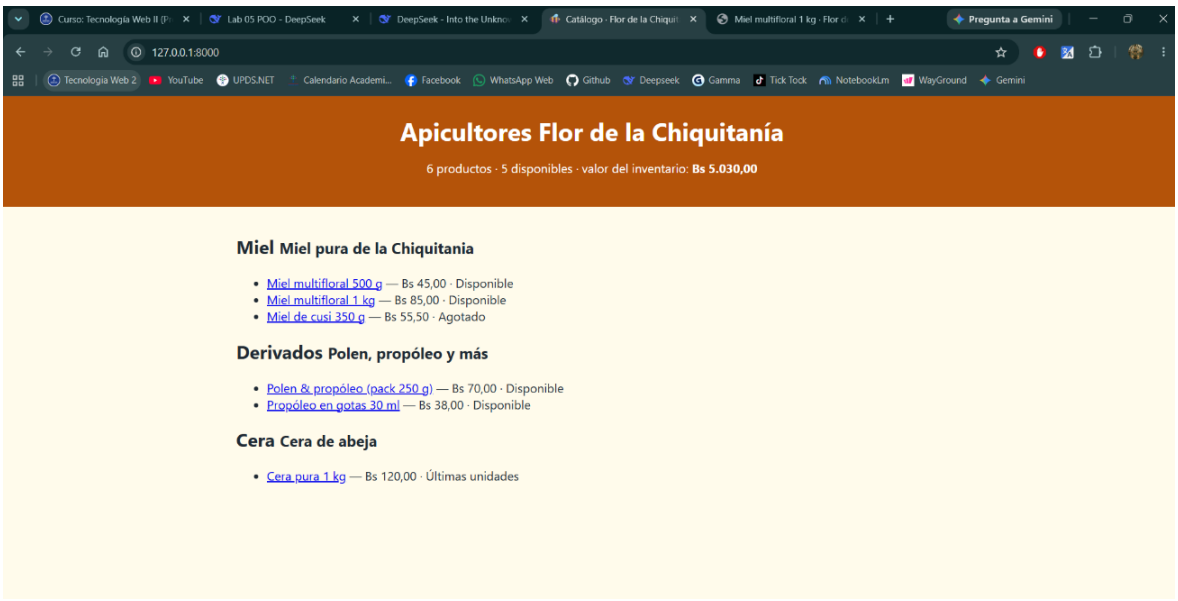
3.1.Laboratorio 6: Poo, Composer y Polimorfismo

El laboratorio 6 organiza el proyecto PHP mediante Composer y autoload PSR-4. El espacio de nombres Apicultores se asocia con el directorio src/, lo que permite cargar clases sin mantener una lista de require manuales. En el modelo, Articulo actúa como clase abstracta; Producto y Servicio heredan de ella y cumplen el contrato de la interfaz Vendible. El trait ConBolivianos reutiliza la funcionalidad de formato monetario, mientras que Categoria representa un conjunto definido de categorías.

El polimorfismo permite que el catálogo invoque precioVenta() sobre distintos objetos sin conocer todos sus detalles internos. La bitácora reporta un descuento del 10 % desde 12 unidades para productos, mientras que los servicios se cobran según su regla de precio. También documenta que un servicio está disponible sin depender de un stock físico.

Resultado consignado en la bitácora: 14 de 14 comprobaciones satisfactorias. Se reporta un catálogo con 6 productos y 2 servicios, 7 elementos disponibles e inventario de Bs 5.030,00 sin contar servicios.

Figura 1. Catálogo del laboratorio 6



Nota. Captura de la bitácora del laboratorio 6 que muestra el catálogo en el navegador.

Figura 2. Verificador del laboratorio 6

```
PS C:\Laboratorio 6\TW2\lab06\inicio> php verificar.php
[OK] composer.json declara PSR-4: Apicultores\ → src/
[OK] El autoload encuentra clases, interfaz, trait y enum por su espacio de nombres
[OK] Las páginas no usan require de src/: solo vendor/autoload.php
[OK] Enum Categoria: from, tryFrom, cases y descripcion()
[OK] Articulo es abstracta: no se puede instanciar
[OK] Producto y Servicio heredan de Articulo e implementan Vendible y JsonSerializable
[OK] Polimorfismo: 2 miel = 90 · 12 miel = 486 (−10 %) · 12 personas al taller = 2400 (sin descuento)
[OK] Producto no vende sin stock; un servicio siempre está disponible
[OK] Trait ConBolivianos: Articulo lo usa y su método es protected
[OK] Catálogo: 6 productos + 2 servicios, 7 disponibles, inventario 5030 (sin servicios)
[OK] totalPedido: 12 MIE-500 + 1 SRV-EXT = 636; código inexistente → DomainException
[OK] json_encode(Producto) usa jsonSerialize con parent:: (tipo, código, categoría, stock)
PHP Deprecated:  Function curl_close() is deprecated since 8.5, as it has no effect since PHP 8.0 in C:\Laboratorio 6\
TW2\lab06\inicio\Verificador.php on line 57

Deprecated: Function curl_close() is deprecated since 8.5, as it has no effect since PHP 8.0 in C:\Laboratorio 6\TW2\l
ab06\inicio\Verificador.php on line 57
[OK] La página muestra categorías del enum, precio por docena y servicios
PHP Deprecated:  Function curl_close() is deprecated since 8.5, as it has no effect since PHP 8.0 in C:\Laboratorio 6\
TW2\lab06\inicio\Verificador.php on line 57

Deprecated: Function curl_close() is deprecated since 8.5, as it has no effect since PHP 8.0 in C:\Laboratorio 6\TW2\l
ab06\inicio\Verificador.php on line 57
[OK] api.php responde JSON con los 8 artículos
Resultado: 14 de 14 comprobaciones
```

Nota. Captura de consola incluida en la bitácora; reporta 14 de 14 comprobaciones.

3.2. Laboratorio 7: Front Controller, Router Y Seguridad De Vistas

El laboratorio 7 centraliza el ingreso de solicitudes en public/index.php. El router identifica la ruta y el método HTTP, y distingue una ruta inexistente (404) de una ruta existente utilizada con un método no permitido (405). La jerarquía de excepciones permite representar errores del dominio, como un conflicto de stock (409) o datos no válidos (422).

Las plantillas escapan automáticamente los textos con htmlspecialchars para reducir el riesgo de XSS. El manejador global registra los detalles técnicos de errores inesperados en un archivo privado y muestra al usuario un mensaje con una referencia, sin revelar rutas ni detalles internos.

Resultado consignado en la bitácora: el comando docker compose exec -T php php verificar.php terminó con 15 de 15 comprobaciones aprobadas.

Figura 3. Verificación del laboratorio 7

```
PS C:\TW2\lab07\inicio> docker compose exec -T php php verificar.php
[OK] Jerarquía: NoEncontrado, Conflicto y DatosInvalidos heredan de ErrorAplicacion (RuntimeException)
[OK] Cada excepción sabe su código HTTP: 404, 405, 409, 422
[OK] Router: parámetros {codigo}, 404 si no existe y 405 (con métodos permitidos) si cambia el método
[OK] Plantilla escapa automáticamente (también en arreglos) y respeta Crudo
[OK] Plantilla rechaza objetos (la vista recibe datos, no modelos) y nombres con ../
[OK] Front controller: / y /producto/MIE-1K (200); el CSS se sirve como archivo
[OK] 404 con página propia para un producto o una ruta inexistente
[OK] POST a / → 405 con cabecera Allow: GET
[OK] Escape automático en la página: la búsqueda <script> se muestra como texto
[OK] /api/articulos devuelve JSON (8 artículos)
[OK] Cotización válida: 12 MIE-500 + 1 SRV-EXT → 200, total Bs 636,00
[OK] Cantidades inválidas → 422, error por campo y lo escrito vuelve escapado
[OK] Pedir 5 CER-1K (hay 4) → 409 con el mensaje de stock
[OK] Error inesperado → 500 amable con referencia, SIN clase, ruta ni traza
[OK] El detalle quedó en datos/errores.log con la misma referencia
Resultado: 15 de 15 comprobaciones
PS C:\TW2\lab07\inicio>
```

Nota. La captura de consola incluida en la bitácora reporta 15/15 comprobaciones.

Figura 4. Vista del catálogo del laboratorio 7



Nota. Captura de la interfaz web del catálogo y búsqueda.

3.3. Laboratorio 8: Mysql, Pdo Y Crud De Productos

El laboratorio 8 traslada el almacenamiento de productos a MySQL y utiliza PDO para ejecutar consultas desde PHP. La conexión se centraliza en los archivos de base de datos y la lógica de consultas se separa en una clase dedicada. Las consultas preparadas permiten enviar los valores como parámetros, en lugar de concatenarlos directamente en la instrucción SQL.

La bitácora describe correcciones en la conexión Docker/PDO y la implementación de operaciones CRUD con manejo de errores de base de datos. También indica que se manejan errores como el código duplicado 1062 y que se ajusta el escape de comodines en búsquedas. El autor registra un resultado final de 15 de 15 comprobaciones.

Figura 5. Listado de productos en MySQL



Nota. Captura del listado de productos de la bitácora del laboratorio 8.

Figura 6. Formulario de registro de producto



Nota. Captura del formulario usado para registrar productos.

Figura 7. Resultado de edición y eliminación



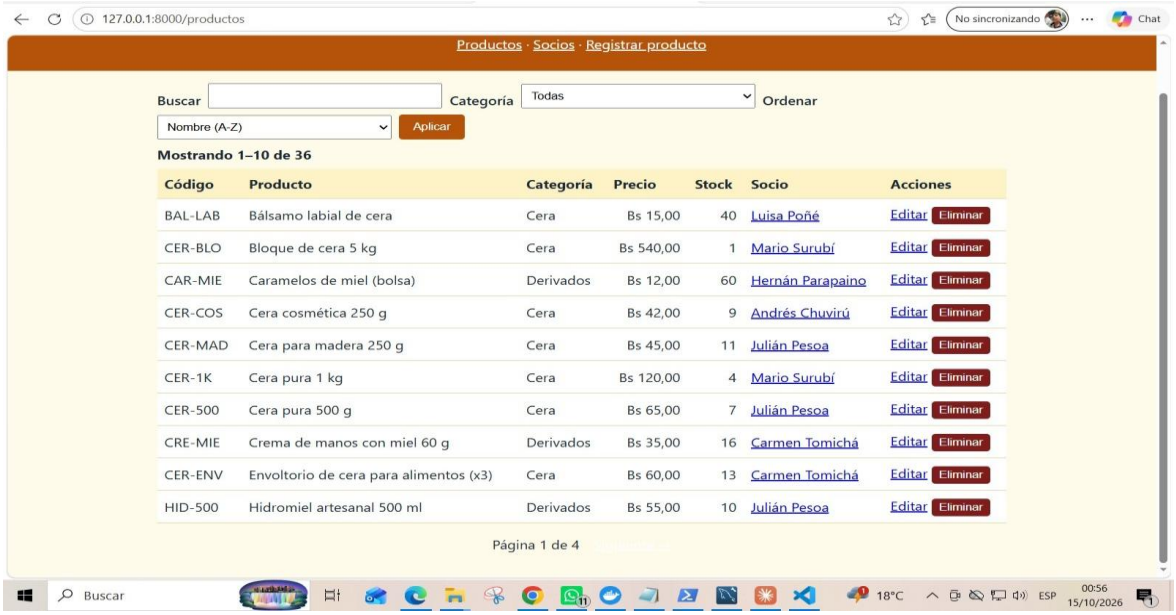
Nota. Captura de la interfaz con un aviso de registro y el listado actualizado.

3.4. Laboratorio 9: Repositorios, Socios, Filtros Y Paginación

El laboratorio 9 profundiza la separación entre controladores y acceso a datos mediante repositorios. La bitácora registra el uso de ProductoRepositorioMysql y SocioRepositorio, consultas por socio, filtros por categoría y texto, ordenamiento por opciones permitidas y paginación con LIMIT y OFFSET.

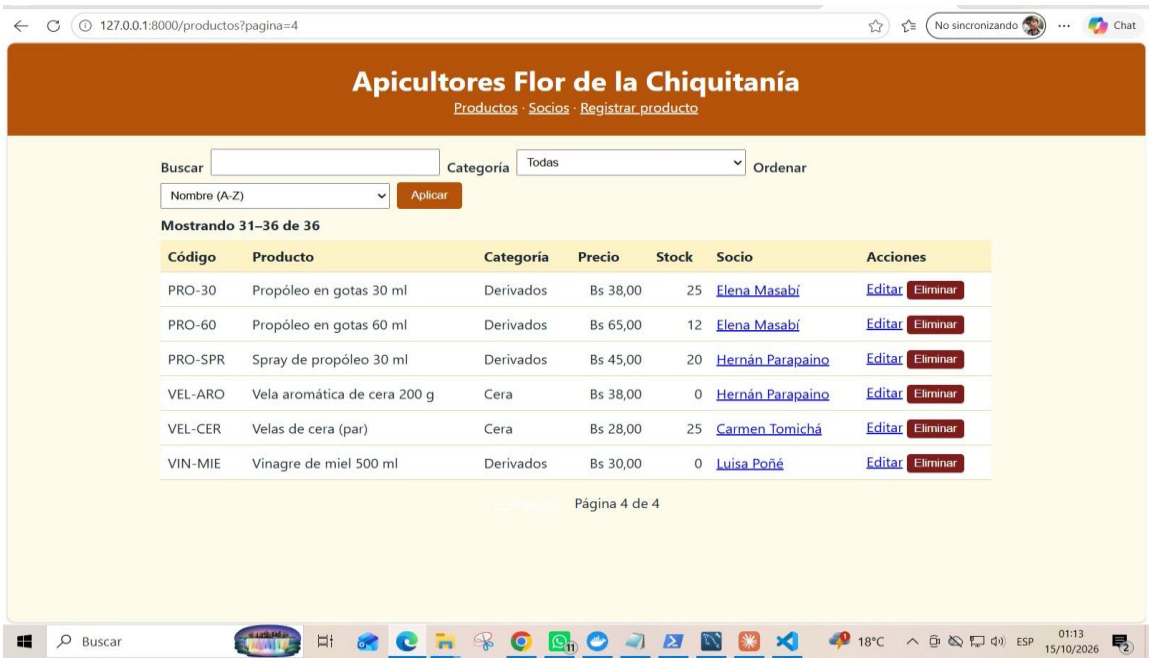
Las capturas muestran un listado de 36 productos dividido en cuatro páginas, la navegación a la página 4, el listado de socios y el detalle de los productos asociados a un socio. También aparece el formulario para registrar un producto. La bitácora informa un resultado final de 14 de 14 comprobaciones, incluidas paginación, filtro, orden, consulta por socio y operaciones de edición/eliminación de productos.

Figura 8. Listado y paginación de productos



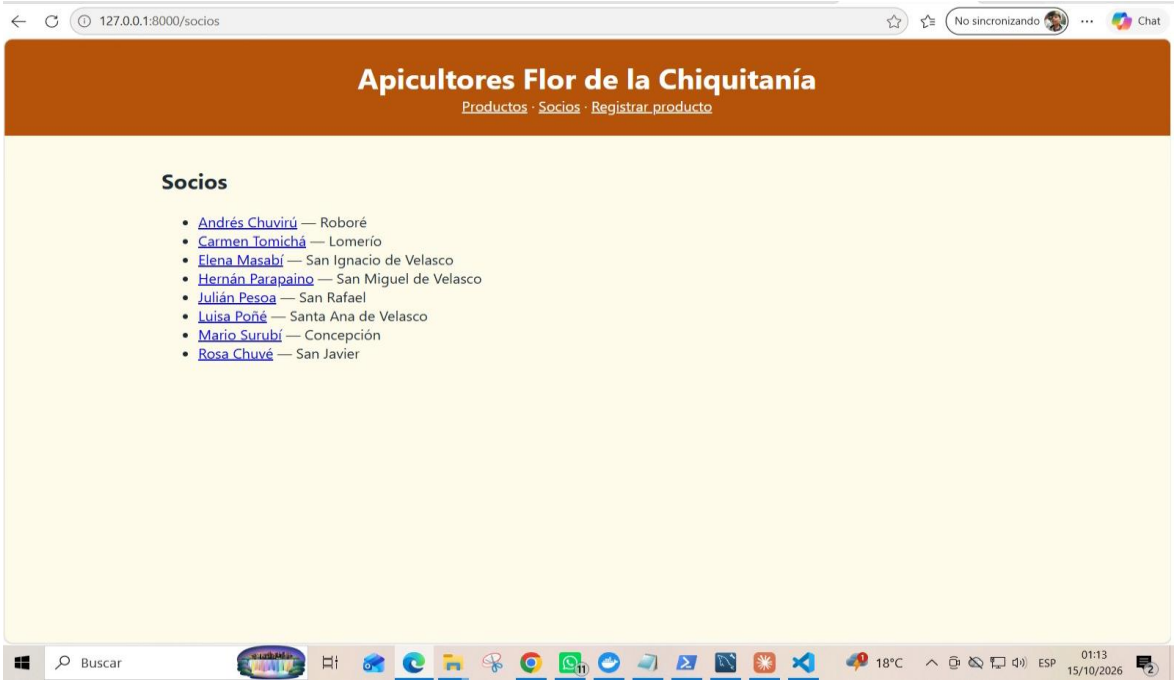
Nota. Captura de la página de productos; se observan filtros, opciones de orden y navegación paginada.

Figura 9. Paginación en la página 4



Nota. Captura de la cuarta página del listado, reportada en la bitácora del laboratorio 9.

Figura 10. Lista de socios



Nota. Captura del listado de socios.

Figura 11. Productos asociados a un socio



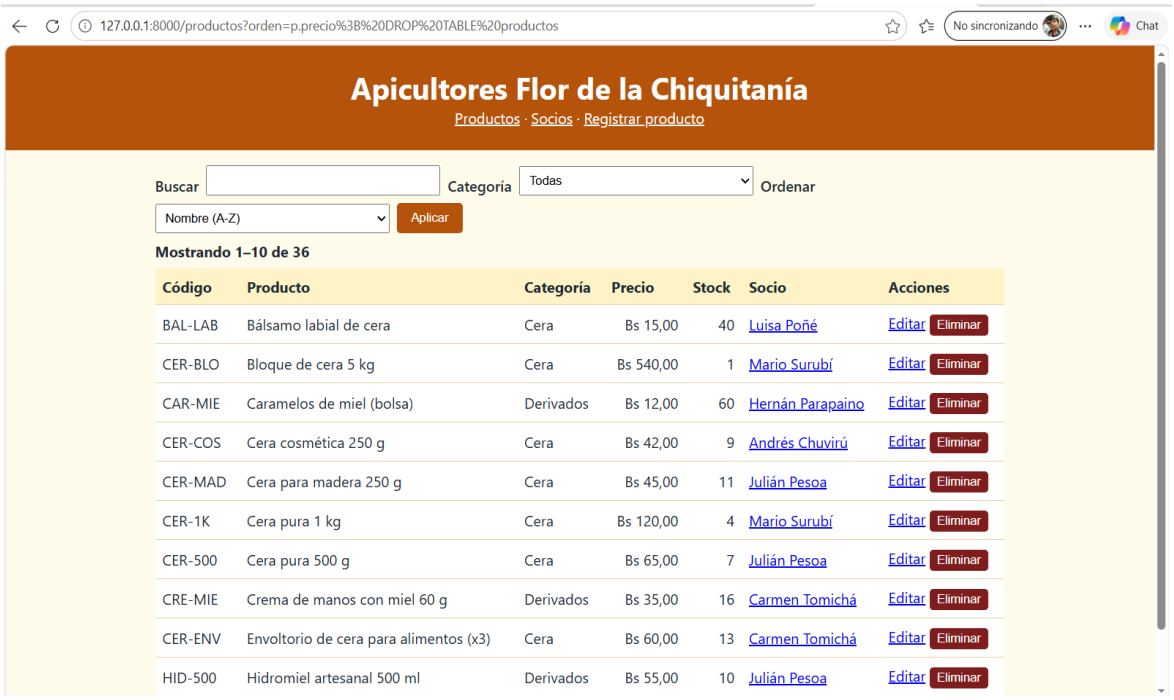
Nota. Captura del detalle de un socio y sus productos.

Figura 12. Verificador del laboratorio 9

```
PS C:\TW2> cd C:\TW2\lab09\inicio
PS C:\TW2\lab09\inicio> code .
PS C:\TW2\lab09\inicio> $Kit = 'C:\TW2\TW2_Aula.ps1'
PS C:\TW2\lab09\inicio> $Curso = 'C:\TW2'
PS C:\TW2\lab09\inicio> powershell -NoProfile -ExecutionPolicy Bypass -File $Kit -Accion Verificar -Laboratorio 9 -Raiz $Curso
29.8.2
El verificador puede restaurar datos de laboratorio. Guarde antes lo que quiera conservar.
[OK] Pagina: 36 resultados de 10 en 10 ÔåÆ 4 p├íginas; rango "Mostrando 11ÔÇô20 de 36"
[OK] Repositorio MySQL devuelve OBJETOS Producto con su Socio
[OK] buscar(): p├ígina 4 trae los 6 ├║ltimos y el total es 36 (LIMIT/OFFSET)
[OK] Filtro por categor├¡a: Cera ÔåÆ 10 productos; Cera + "vela" ÔåÆ 2
[OK] Orden por precio: el m├ís barato es CAR-MIE (Bs 12) y el m├ís caro CER-BLO (Bs 540)
[OK] deSocio(4) ÔåÆ los 6 productos de Juli├ín Pesoa
[OK] Interfaz: el repositorio en MEMORIA responde igual que el de MySQL
[OK] P├ígina 1: 10 filas, "Mostrando 1ÔÇô10 de 36", enlace Siguiente y sin Anterior
[OK] Los enlaces de paginaci├│n conservan los filtros (categor├¡a y orden)
[OK] orden desconocido ÔåÆ orden por nombre (lista permitida); pagina=-5 ÔåÆ p├ígina 1
[OK] pagina=abc ÔåÆ 1 ┬À pagina=99 ÔåÆ la ├║ltima (4)
[OK] Socio 3 (Elena Masab├¡): sus 4 productos y el valor en stock
[OK] CRUD con el repositorio: registrar ÔåÆ 303 y aparece en el socio 4 (7 productos)
[OK] Editar y eliminar siguen funcionando (303)
Resultado: 14 de 14 comprobaciones
Resultado guardado en C:\TW2\lab09\inicio\evidencias\verificacion-20261015-003721.txt
```

Nota. Captura de consola incluida en la bitácora; indica resultado de 14 de 14 comprobaciones.

Figura 13. Prueba de seguridad con entrada de búsqueda



Nota. Captura identificada en la bitácora como intento de inyección SQL.

4. ELEMENTOS TÉCNICOS COMUNES

4.1. Acceso a Datos con PDO

PDO proporciona una interfaz para acceder a MySQL desde PHP. La conexión debe configurarse con el DSN correcto y credenciales externas al directorio público. Los errores de conexión o consulta deben gestionarse mediante excepciones y mensajes seguros. La bitácora del laboratorio 8 señala que se desactivó la emulación de consultas preparadas en la configuración utilizada.

4.2. Consultas preparadas e inyección SQL

La inyección SQL aparece cuando entradas no confiables se incorporan directamente a una consulta. Con prepare() y execute(), la instrucción SQL se define por separado de los valores. Para ordenar resultados, el nombre de columna o criterio no debe recibirse libremente desde la URL; debe seleccionarse de una lista permitida. La prueba de seguridad se debe realizar en el entorno local de la práctica.

4.3. Validación, integridad y errores

La validación de formularios debe complementarse con restricciones de la base de datos, como claves primarias, claves foráneas, campos obligatorios y unicidad de códigos. Los errores técnicos deben registrarse internamente, mientras que la interfaz muestra mensajes claros que no expongan secretos. Los filtros y parámetros de paginación también deben validarse.

5. RESUMEN DE RESULTADOS DOCUMENTADOS

Tabla 1. Resultados de las comprobaciones por laboratorio y evidencias incorporadas.

Laboratorio	Resultado consignado en bitácora	Evidencia incorporada
06	14/14 comprobaciones aprobadas	Figuras 1–2
07	15/15 comprobaciones aprobadas	Figuras 3–4
08	15/15 comprobaciones aprobadas	Figuras 5–7
09	14/14 comprobaciones aprobadas	Figuras 8–13

Nota. Estos resultados se transcriben de las bitácoras entregadas.

6. ANÁLISIS

Los laboratorios muestran una progresión: primero se organiza el modelo con conceptos de POO; después se centraliza el enrutamiento y se mejora el manejo de errores; posteriormente se incorpora MySQL con PDO; finalmente se agregan repositorios, relaciones con socios, filtros y paginación. Esta separación facilita mantener la lógica del dominio y la presentación sin concentrar todas las responsabilidades en un solo archivo.

Las capturas de los laboratorios 8 y 9 muestran operaciones de interfaz de productos y consultas relacionadas con socios. No obstante, las evidencias visuales de la bitácora del laboratorio 8 se concentran principalmente en productos.

La captura del laboratorio 9 identificada como intento de inyección SQL muestra una entrada de búsqueda especial.

7. CONCLUSIONES

La utilización de Composer y PSR-4 permitió organizar mejor las clases y automatizar su carga, evitando utilizar múltiples instrucciones manuales. Esto facilita el mantenimiento y la ampliación de la aplicación. **Alejandro Fabián Cabrera Flores**

El uso de clases abstractas, interfaces, traits y polimorfismo permitió representar productos y servicios con diferentes comportamientos dentro de un mismo catálogo, favoreciendo la reutilización del código y una mejor organización. **María Alejandra Suárez O.**

La implementación del front controller, el router y las excepciones propias permitió gestionar las solicitudes de manera centralizada y responder con códigos HTTP adecuados. Además, el escape de las plantillas y el manejo seguro de errores contribuyeron a mejorar la seguridad de la aplicación. **Jade Guissel Flores Coria**

La integración de MySQL mediante PDO y consultas preparadas permitió separar los datos ingresados por los usuarios de las instrucciones SQL, reduciendo el riesgo de inyección SQL y mejorando la seguridad en el acceso a la base de datos. **Ariel Salas Tococari**

Los repositorios, filtros y mecanismos de paginación facilitaron la organización y consulta de los datos. Asimismo, las bitácoras registran resultados satisfactorios en los verificadores de los cuatro laboratorios, respaldando el trabajo realizado mediante evidencias de las pruebas. **Renzo Molina Navia**

8. RECOMENDACIONES

1. Mantener las credenciales fuera de public/ y evitar exponer información sensible en capturas o repositorios.**Alejandro Fabián Cabrera Flores**

2. Utilizar consultas preparadas con PDO para proteger las operaciones CRUD contra la inyección SQL**. María Alejandra Suárez O.**

3. Validar los filtros, la paginación y los criterios de ordenamiento para garantizar consultas seguras. **Jade Guissel Flores Coria**

4. Conservar los resultados de los verificadores y el código correspondiente a la versión final del proyecto. **Ariel Salas Tococari**

5. Adjuntar evidencias del CRUD de socios y de las pruebas de inyección SQL realizadas. **Renzo Molina Navia**

REFERENCIAS

Composer. (s. f.). Autoloading. https://getcomposer.org/doc/01-basic-usage.md#autoloading

MySQL. (s. f.). MySQL 8.4 Reference Manual. https://dev.mysql.com/doc/refman/8.4/en/

OWASP Foundation. (s. f.). SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

PHP Documentation Group. (s. f.). PDO::prepare. PHP Manual. https://www.php.net/manual/en/pdo.prepare.php

PHP Documentation Group. (s. f.). Connections and connection management. PHP Manual. https://www.php.net/manual/en/pdo.connections.php

Universidad Privada Domingo Savio. (2026a). Guía de laboratorio 06: Productos y servicios: Composer, herencia, interfaces, traits y enums [Material de curso]. Tecnología Web II.

Universidad Privada Domingo Savio. (2026b). Guía de laboratorio 07: Front controller, router, excepciones propias y plantillas con escape automático [Material de curso]. Tecnología Web II.

Universidad Privada Domingo Savio. (2026c). Guía de laboratorio 08: Productos en MySQL 8.4 con PDO: CRUD con consultas preparadas [Material de curso]. Tecnología Web II.

Molina Navia, R. (2026, 18 de octubre). Bitácora de IA — Laboratorio 9. Tecnología Web II.

Salas Tococari, A. (2026, 18 de octubre). Bitácora de IA — Laboratorio 7. Tecnología Web II.

Suárez Obaldo, M. A. (2026, 17 de octubre). Bitácora de IA — Laboratorio 8. Tecnología Web II.

Bitácora de IA del laboratorio 6. (2026). Tecnología Web II.